Executable Governance

IT Strategy — ista SE

March 2026

Agenda

1. What is IT Governance?
2. Why Traditional Governance Fails
    - Problem 1: Siloed, inconsistent policies
    - Problem 2: Stale and invisible policies
    - Problem 3: Stale, disconnected data
    - Problem 4: Manual, inconsistent controls
3. The Way Forward

Chapter 1

The Core Problem

Organizations make thousands of technology decisions every day.

Without constraints, systems drift.

Drift creates:

- **Risk** — security exposure, compliance violations
- **Complexity** — duplicated systems, inconsistent patterns
- **Misalignment** — IT decisions disconnected from business intent

What Governance Does

IT Governance keeps technology decisions aligned with business objectives through two mechanisms:

Mechanism	Purpose	Examples
Policies	Define the boundaries	"Databases must be encrypted at rest"
Controls	Measure and enforce compliance	Automated scanner, approval gate

> *The goal of governance is not to slow things down.*
> *It is to make the **right thing** the **easy thing**.*

Example Policies

Domain	Policy
Ownership	Every application must have a defined business owner and cost center
Resilience	Mission-critical systems must achieve an RTO of ≤1 hour
Network Security	Public endpoints must be protected by a WAF
Data Security	Databases must use encryption at rest and in transit
Identity	All application access must be MFA-enabled
Device	End-user devices must enforce approved software and secure configuration

Chapter 2

The Root Causes

Traditional governance fails in **predictable ways**, always for the same reasons:

```
Siloed ownership  ──►  Inconsistent policies  ──►  Confusion, ignored rules
                                                          │
Document-based  ──►  Stale policies  ──────────►  Compliance gaps
                                                          │
Manual data  ──►  Wrong data  ──────────►  Wrong decisions
                                                          │
Human gatekeepers  ──►  Bottlenecks  ──────────►  Routes around process
```

Problem 1: Policies are Siloed and Inconsistent

Multiple disciplines define policies **independently**, with no structural incentive to align:

- Enterprise Architecture — structure and evolution
- Security Governance — risk and control objectives
- Platform Governance — technical guardrails
- Data Governance — classification and usage
- Financial Governance — cost accountability

The result: overlapping, contradictory, or simply unknown policies published through different channels to different audiences.

Consequence: Stakeholders receive conflicting signals and learn to ignore governance entirely.

Fix: A single authoritative policy registry with cross-domain ownership.

Problem 2: Policies Become Stale and Invisible

Policy documents are written once and forgotten. There is no structural incentive to keep them current. Stakeholders are rarely notified when policies change.

Consequence: Teams operate on outdated policies without knowing it. Compliance measured against the wrong baseline is meaningless.

Fix: Policies defined as code, versioned in a repository, with changes surfaced like software changes — diffs, changelogs, automated notifications.

Problem 3: Governance Data is Stale or Disconnected

Controls are only as good as the data they operate on:

System	Data
CMDB	Asset inventory, ownership
LeanIX	Application portfolio, architecture
Azure Resource Manager	Infrastructure configuration
Defender for Cloud / Tenable	Security posture
Intune / Jamf	Device compliance

Data is maintained manually, in isolation, with inconsistent identifiers across systems.

Problem 3: A Concrete Example

KPI dashboards for ista's 10 golden rules of IT security are generated from LeanIX data.

```
Stale LeanIX data
      │
      ▼
Incorrect KPI  ──►  Teams investigate non-issues
               ──►  Real issues go undetected
               ──►  Management trust in governance erodes
```

Consequence: Governance decisions are made against a snapshot of reality that no longer exists.

Fix: Canonical identifiers enforced at ingestion (e.g. deployment fails if Azure `application` tag doesn't match a LeanIX application name). API-driven integration, not manual exports.

Problem 4: Controls are Manual and Inconsistently Applied

Controls implemented as human approval gates fail in two ways:

Gatekeepers become bottlenecks

- Committees overloaded → decisions delayed
- Teams learn to route around the process

Approvals expire silently

- A configuration is approved at time T
- Six months later it may no longer be compliant
- Nobody checks

Consequence: Governance is applied inconsistently. Non-compliant configurations

Summary

Problem	Root Cause	Fix
Inconsistent policies	No incentive to align	Policy registry + cross-domain ownership
Stale policies	Document-based	Policies as code, versioned
Stale data	Manual, siloed maintenance	Canonical IDs enforced at source
Manual controls	Human gatekeepers	Automate decisions, committees for exceptions

> ***The question is not whether to automate governance.***
> ***It's where to start.***

Next

The Modern Model: Executable Governance

- Connecting the data
- Enforcing at the platform level
- Policy-as-Code
- Continuous compliance reporting
- Golden Paths

[draft — next session]